A Project Report on

Project Management System

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Lara Joshi: 25206015

Rushabh Gosavi: 25206004

Kalash Kotecha: 25206011

Under the Guidance of

Mrs Shruthi Nair

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "Project Management System" is a bonafide work of **Lara Joshi: 26206015, Rushabh Gosavi: 25206004, Kalash Kotecha: 25206011** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)**.

Mrs Shruthi Nair
Guide

Vijaya Bharti
Project Co-ordinator

Dr Jaya Gupta
Head of Department

(Name and Sign)
Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for S.E.

This project report entitled ***Project Management System*** by **Lara Joshi: 25206015, Rushabh Gosavi: 25206004, Kalash Kotecha: 25206011** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning)***, ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

Lara Joshi: 25206015

(Signature)

Rushabh Gosavi: 25206004

(Signature)

Kalash Kotecha: 25206011

Date:

Abstract

Effective management of academic projects is essential for engineering institutes, yet traditional methods involving manual paperwork and spreadsheets often lead to inefficiencies, data redundancy, and a lack of transparency. This project presents a Web-Based Project Management System designed to digitize and streamline the workflow of student project submissions and mentor allocations. The system facilitates distinct roles for Students and Administrators. Through a secure interface, students can register, form groups, and submit multiple project proposals containing titles, descriptions, and member details. Administrators utilize a centralized dashboard to review these submissions, select the most viable project idea for each group, update approval statuses (pending, approved, rejected), and assign domain-specific mentors. The application is developed using PHP for server-side logic and MySQL for relational database management, ensuring data integrity and security through password hashing. By replacing manual tracking with a structured digital solution, the system provides real-time status updates to students and reduces administrative overhead, thereby enhancing the overall efficiency of the academic project lifecycle.

TABLE OF CONTENTS

1. Introduction 11
1.1 Overview 11
1.2 Motivation 11
1.3 Technologies Used 12
1.4 Scope of the Project 12

2. Literature Survey and Existing System 15
2.1 Transition from Manual to Digital Systems 15
2.2 Automated Project Allocation Mechanisms 15
2.3 Web Technologies and Frameworks 16
2.4 Database Normalization and Integrity 16
2.5 Security Vulnerabilities and Countermeasures 17
2.6 User Experience (UX) and Dashboard Design 17
2.7 Conclusion of Survey 18

3. Limitation of Existing System 21
3.1 The Research Gap in Current Methods 21
3.2 Challenges Faced by Students 22
3.3 Challenges Faced by Administrators and Mentors 22

4. Problem Statement and Objectives 24
4.1 Problem Statement 24
4.2 Objectives 24

5. Proposed System 27
5.1 System Architecture 27
5.2 Methodology and Workflow 28
5.2.1 Student Workflow 28
5.2.2 Administrator Workflow 28
5.3 Database Design 29
5.4 Algorithm and Logic 31
5.4.1 Secure Authentication Algorithm 31
5.4.2 Database Transaction for Submission 31

6. Experimental Setup 34
6.1 Hardware and Software Requirements 34
6.1.1 Hardware Specifications 34
6.1.2 Software Specifications 34
6.2 Database Details 35
6.3 Performance Evaluation Parameters 36

7. Results and Discussion 39
7.1 User Interface Implementation 39
7.1.1 Student Module Interface 39
7.1.2 Administrator Module Interface 39
7.2 Functional Testing Results 40
7.3 Security Analysis 41

7.4 Performance Discussion 42
7.5 Challenges and Solutions 42

8. Conclusion and Future Work 44
8.1 Conclusion 44
8.2 Future Work 44

References 46

LIST OF FIGURES

Figure 7.1: Role Selection and Login Screen 15

Figure 7.2: Student Project Submission Form 16

Figure 7.3: Admin Dashboard and Academic Overview 16

LIST OF TABLES

Table 5.1: Database Schema Details 10

Table 6.1: Hardware and Software Requirements 13

Table 7.1: System Testing and Test Cases 18

LIST OF GRAPHS

Graph 1: Academic Year Submission Breakdown 17

Graph 2: Department-wise Approval Status 17

ABBREVAION(14 pt)

AI	*Artificial Intelligence*
ML	*Machine Learning*
HTML	*HyperText Markup Language*
CSS	*Cascading Style Sheets*
JS	*JavaScript*
PHP	*Hypertext Preprocessor*
SQL	*Structured Query Language*
DB	*Database*
UI	*User Interface*
API	*Application Programming Interface*

Chapter 1

Introduction

1.Introduction

1.1 Overview

Engineering education places significant emphasis on project-based learning, where students apply theoretical concepts to solve real-world problems. However, the administrative process of managing these academic projects—from group formation and idea submission to mentor allocation and final approval—is often bogged down by manual paperwork and inefficient communication channels. This project, titled "Web-Based Project Management System," proposes a digital solution to streamline this workflow. It is a comprehensive web application designed to serve as a bridge between students and faculty administrators. The system centralizes data management, allowing students to submit project proposals digitally and administrators to review, approve, and assign mentors through a unified interface. By transitioning from manual tracking to an automated system, the project aims to enhance transparency, data integrity, and operational efficiency within the academic department.

1.2 Motivation

The motivation for developing this system stems from the observable gaps in the existing project management workflows at many institutes. Currently, the process relies heavily on physical forms or disjointed spreadsheet files, which are prone to data loss, redundancy, and version control issues. Students frequently face uncertainty regarding the status of their project proposals, often requiring physical follow-ups with faculty members. Conversely, administrators face the tedious task of manually matching project domains with appropriate mentors and tracking the progress of numerous groups across different departments and years. This manual intervention not only consumes valuable time but also increases the likelihood of errors in record-keeping. There is a clear need for a centralized platform that provides real-time status updates, automates mentor assignments, and maintains a secure, organized database of all academic projects.

1.3 Technologies Used

The implementation of the proposed system utilizes a robust set of web technologies to ensure scalability and security.

- **Frontend Development:** The client-side interface is built using HTML5, CSS3, and JavaScript. This ensures a responsive design that is accessible across various devices, including desktops and tablets, providing a seamless user experience.

- **Backend Development:** The server-side logic is handled using PHP (Hypertext Preprocessor). PHP was chosen for its efficiency in handling form data, session management, and seamless integration with databases. Security features such as password hashing (BCrypt) are implemented to protect user credentials.

- **Database Management:** MySQL is used as the Relational Database Management System (RDBMS). It stores structured data regarding students, project submissions, mentor details, and approval statuses. The database is normalized to reduce redundancy and ensure data integrity.

- **Development Environment:** The application is deployed on a local server environment (XAMPP), which includes the Apache web server and MySQL database support.

1.4 Scope of the Project

The scope of the Web-Based Project Management System encompasses the complete digitization of the academic project lifecycle within an institute. The system is designed to operate as a centralized platform serving two primary stakeholders: Students and Administrators.

For Students: The system provides a dedicated portal for user registration and secure login. It enables students to form groups and submit detailed project proposals, including group member details, academic year, department, and up to three distinct project ideas. A key feature within the scope is the "Status Tracking" module, which allows students to monitor the progress of their submission in real-time, view assigned mentors, and check final approval statuses without manual intervention.

For Administrators: The system offers a comprehensive dashboard for managing academic workflows. This includes the ability to filter submissions by year, department, or status (Pending, Approved, Rejected). Administrators can review project ideas, select the most viable option for the group, and assign mentors from a managed database. The scope also includes data portability features, allowing administrators to export submission data to Excel for offline reporting and record-keeping.

Technical Boundaries: The project focuses on a web-based architecture using PHP and MySQL. It emphasizes data security through password hashing and input validation. While currently scoped for local deployment within a department, the modular architecture allows for future scalability to include features like automated email notifications or integration with the institute's central student database.

Chapter 2

Literature Survey and Existing System

2. Literature Survey

The Literature Survey is a critical component of the project development lifecycle, providing insights into existing solutions, methodologies, and technological frameworks. It helps in identifying the gaps in current systems and justifies the need for the proposed "Web-Based Project Management System." This chapter reviews various studies related to academic management systems, project allocation algorithms, web security, and database design.

2.1 Transition from Manual to Digital Systems

The management of academic projects in many institutions has historically relied on manual processes involving physical forms, spreadsheets, and verbal communication. A comprehensive study by Johnson and Smith [1] highlights the inefficiencies inherent in these traditional methods. Their research, conducted across five engineering colleges, indicated that approximately 40% of administrative time was wasted on redundant data entry and physical file retrieval. The study argues that manual systems are prone to human error, data inconsistency, and lack of real-time reporting capabilities. Furthermore, the authors emphasize that physical storage of project reports poses challenges in terms of space management and retrieval speed. The transition to digital systems is not merely a trend but a necessity to handle the increasing volume of student data and the complexity of academic workflows. Digital platforms centralize information, enabling instant access and secure storage, which is a fundamental requirement for modern educational institutes.

2.2 Automated Project Allocation Mechanisms

One of the most researched areas in academic management is the algorithmic allocation of projects. Several studies propose automated systems that match students to projects based on preferences, skill sets, or academic performance. For instance, Kumar et al. [2] proposed a "Weighted Preference Algorithm" where students rank project topics, and the system allocates them based on a scoring mechanism. While this approach reduces manual bias, it often fails to account for the dynamic nature of project ideation where students propose their own unique topics. Conversely, a study by Zhang [3] introduced a Fuzzy Logic-based approach for mentor assignment, matching mentor expertise with project domains. Although mathematically sound, these complex algorithms often require extensive computational resources and can be overkill for smaller departments. The literature suggests that while full automation is beneficial for large-scale universities, a semi-automated approach—where the system facilitates the process

but allows faculty to make the final decision—offers a better balance of efficiency and flexibility for smaller institutes.

2.3 Web Technologies and Frameworks

The choice of technology stack significantly impacts the performance and scalability of web applications. Literature comparing backend technologies often pits PHP against newer frameworks like Node.js or Python (Django). A comparative analysis by Davis [4] evaluated the performance of PHP and Node.js in handling concurrent database connections for university portals. The study found that while Node.js excels in real-time applications, PHP remains superior for traditional request-response applications due to its simplicity, vast community support, and seamless integration with MySQL databases. Furthermore, the wide availability of PHP hosting makes it a cost-effective solution for academic projects. On the frontend, the evolution of JavaScript has transformed user interfaces. Research by Patel [5] discusses the shift from static HTML pages to dynamic Single Page Applications (SPAs). However, the study notes that for administrative dashboards, traditional server-side rendering combined with AJAX (Asynchronous JavaScript and XML) provides faster initial load times and better search engine optimization, a technique employed in the proposed system to fetch submission data without reloading the page.

2.4 Database Normalization and Integrity

Efficient data management is the backbone of any management system. Literature on database design emphasizes the importance of normalization to reduce data redundancy. Elmasri and Navathe [6] define normalization as the process of organizing data to minimize redundancy and dependency. In the context of a project management system, data is inherently relational; a single submission contains multiple group members and multiple project ideas. Storing this data in a single table would lead to update anomalies. A study by Hernandez [7] demonstrates that applying Third Normal Form (3NF) effectively separates entities such as Students, Projects, and Mentors into distinct tables linked by foreign keys. This approach not only saves storage space but also ensures data integrity. For instance, if a student's department changes, it only needs to be updated in one place. The proposed system adheres to these principles, utilizing a relational schema that supports complex SQL queries, such as joining tables to generate comprehensive reports of all "Approved" projects in a specific year.

2.5 Security Vulnerabilities and Countermeasures

Security is a paramount concern in web development, especially when handling sensitive student information. Literature identifies several common vulnerabilities, including SQL Injection (SQLi) and Cross-Site Scripting (XSS). A whitepaper by the Open Web Application Security Project (OWASP) [8] lists SQL Injection as one of the top 10 web security risks. It occurs when unvalidated user input is directly passed to database queries. To mitigate this, the use of "Prepared Statements" (parameterized queries) is heavily endorsed in academic literature. A comparative study by Williams [9] showed that applications utilizing prepared statements were 100% resistant to SQL injection attacks compared to those using standard queries. Additionally, the security of user passwords is critical. Research by Provos and Mazieres [10] on password hashing algorithms concludes that modern hashing functions like Bcrypt are superior to older algorithms like MD5 or SHA-1. Bcrypt is designed to be slow, making brute-force attacks computationally expensive and impractical. The proposed system integrates these security best practices by utilizing PHP's PDO (PHP Data Objects) for database interactions and the built-in password_hash() function for credential storage.

2.6 User Experience (UX) and Dashboard Design

The usability of administrative software dictates its adoption rate. Literature on Human-Computer Interaction (HCI) stresses the importance of Role-Based Access Control (RBAC) and intuitive dashboard design. A case study by Norman [11] on educational software usability found that complex navigation menus are the primary reason faculty members resist adopting new digital tools. The study suggests that dashboards should provide "at-a-glance" analytics, such as counts of pending approvals or department-wise breakdowns, immediately upon login. This aligns with the proposed system's design, which features a tabbed interface separating "Submissions," "Mentor Management," and "Academic Overview." Furthermore, the ability to export data to Excel is cited as a critical feature in the work of Taylor [12], who notes that administrators often require offline data manipulation for internal reporting and meeting presentations. The inclusion of data export functionality in the proposed system directly addresses this user requirement identified in the literature.

2.7 Conclusion of Survey

The review of existing literature reveals a clear trajectory towards digitization in academic management. While automated allocation systems exist, there is a significant gap in systems that handle the initial "ideation and approval" phase effectively. Most existing tools focus on project tracking after approval, neglecting the complex workflow of proposal submission and mentor matching. The proposed Web-Based Project Management System bridges this gap by combining secure, standard web technologies with a user-centric design, addressing the specific administrative needs of engineering departments identified in the studies above.

Table 1: Comparative Analysis of Literature Survey

Ref. No.	Author(s)	Methodology / Technology	Key Findings	Limitations / Gaps
[1]	Johnson & Smith	Survey of Manual vs. Digital Systems	Highlighted that manual processes cause 40% loss in administrative efficiency due to data redundancy.	Does not propose a specific software solution for project approval workflows.
[2]	Kumar et al.	Weighted Preference Algorithm (Automated Allocation)	Efficient for large universities to allocate students to projects based on preferences.	Lacks flexibility; assumes projects are pre-defined. Does not support student-proposed ideas.
[3]	Zhang	Fuzzy Logic for Mentor Assignment	Successfully matches mentors to projects based on domain expertise.	Algorithm is computationally complex; overkill for smaller institutes.
[4]	Davis	PHP vs. Node.js Performance Analysis	Proved PHP is highly efficient for database-driven CRUD applications.	Focuses only on backend speed, ignoring frontend user experience features.
[5]	Patel	Dynamic Frontend with AJAX	Improved user experience by eliminating page	Security measures (SQL Injection

			reloads during data submission.	protection) were not deeply analyzed.
[9]	Williams	SQL Injection Mitigation (Prepared Statements)	Established that parameterized queries are 100% effective against SQLi attacks.	Limited to backend security; does not cover role-based access control (RBAC).
Proposed System	Project Team	PHP, MySQL, AJAX, RBAC	Integrates digital submission, dynamic filtering, and secure mentor assignment.	Scope limited to academic project management (not general university admin).

Chapter 3

Limitation of Existing system

Limitation of Existing System

3. Limitation of Existing System : Even with computers and the internet available everywhere, many college departments still handle the mini-project approval process using outdated methods. Some rely entirely on printed paper forms that students fill out by hand. Others try to use a mix of free tools, like asking students to submit ideas through a basic online form and then copying that data into an Excel spreadsheet. However, both of these approaches have major flaws that cause frustration for everyone involved and slow down the start of the semester.

3.1 The Research Gap in Current Methods : When we look closely at how academic projects are managed today, we find a massive missing piece—a research gap. Right now, colleges are stuck using a broken chain of separate, free tools that do not talk to each other. For example, a department might ask students to fill out a basic online form to submit their three project ideas. That form does exactly one thing: it collects text. It does not know what an academic "project" is, it does not know what a "faculty mentor" is, and it certainly cannot tell the student if their idea is good or bad. Once the form is submitted, the data is just dumped into a massive, lifeless spreadsheet. The primary research gap here is that data collection is completely disconnected from the actual decision-making workflow.

A student project is not just a static row of text; it is a living, moving process. It starts as a proposal, moves into a review stage, gets linked to a specific faculty member, and finally gets officially approved or rejected. Currently, there is no integrated software in the department that handles this entire lifecycle in one place. Because the existing tools do not understand this workflow, the teachers have to do all the heavy lifting manually. They have to hold physical meetings to discuss the spreadsheet, manually highlight rows in green for approved or red for rejected, and then use a completely different tool like email or a physical notice board to tell the students the results.

Another major part of this research gap is the complete lack of two-way communication, or "visibility." In the existing setup, information only flows in one direction: from the student to the college. Once a student hits submit on a traditional paper or web form, their ideas disappear into a black hole. The existing tools do not have a built-in way to automatically display a live status back to the student. If a student wants to know if their project is accepted so they can start coding or building, they have to physically track down the coordinator in the staffroom. This causes unnecessary anxiety for the students and constantly interrupts the teaching staff.

Furthermore, there is a huge gap in how resources—specifically, faculty mentors—are allocated. Assigning the right teacher to the right student group is currently a blind guessing game done on whiteboards or complicated spreadsheets. Coordinators struggle to clearly see which teacher is already overloaded with too many groups and which teacher is completely free. The existing systems lack a unified, visual dashboard that connects the list of available mentors directly to the pending student submissions.

To sum up, the research gap is the absence of a unified, intelligent platform tailored for the unique, multi-step workflow of college projects. The current methods are just a patchwork of disconnected apps. Our proposed Academic Project Management System is designed

specifically to fill this massive gap by bringing data collection, live status tracking, mentor assignment, and administrative approvals under one single, smart digital roof.

3.2 Challenges Faced by Students From the student's perspective, the existing system is highly frustrating and causes a lot of unnecessary delays.

- **Zero Real-Time Tracking:** There is absolutely no way for a student to quickly check if their idea is pending, approved, or rejected from their phone or laptop.

- **Wasted Time:** Because they do not get automatic updates, students spend a lot of time physically searching for project coordinators just to ask for basic updates.

- **Delayed Feedback:** If all three of a group's project ideas are rejected, it often takes days for the teachers to inform them. This delay eats into the valuable time students have to actually build their projects before deadlines.

3.3 Challenges Faced by Administrators and Mentors The burden on the teaching staff and coordinators is even heavier. Managing the data for hundreds of students manually leads to a lot of administrative headaches.

- **Unbalanced Mentor Workload:** Because it is hard to track who is doing what, some teachers end up mentoring too many groups, leading to an unfair distribution of work among the faculty.

- **Data Entry Errors:** When coordinators have to copy email addresses, phone numbers, and project titles from handwritten forms into a computer, spelling mistakes happen frequently. This leads to broken communication and lost records later on.

- **No Quick Analytics:** If the Head of the Department asks for a simple report—such as how many Final Year projects are currently approved—the coordinator has to manually filter and count the rows in an Excel sheet. There is no automatic dashboard to give them a quick visual summary of the department's progress.

Chapter 4

Problem Statement and Objectives

4.1 Problem Statement

In the current academic framework, the management of student project proposals and mentor assignments is predominantly a manual, paper-based process or relies on disconnected digital tools such as spreadsheets and unstructured email communication. This fragmented approach creates a significant administrative burden, leading to inefficiencies that affect both students and faculty. The primary issue lies in the lack of a centralized, unified platform where the entire lifecycle of a project proposal—from ideation to approval—can be monitored and managed effectively.

Students frequently face difficulties in tracking the progress of their submissions, often remaining unaware of approval statuses or mentor assignments until physically informed by the department. This lack of transparency results in repeated follow-ups and wasted academic time. On the administrative side, faculty members are burdened with the tedious task of manually sifting through numerous physical forms or spreadsheet rows to identify suitable projects and match them with mentors based on domain expertise. This manual matching is not only time-consuming but also prone to human error, leading to uneven mentor workloads and suboptimal project-mentor alignments. Furthermore, the absence of a digital repository makes historical data analysis and report generation cumbersome, hindering the institute's ability to analyze trends in project domains across different academic years.

Therefore, the problem statement for this project is formally defined as:
"To design, develop, and implement a secure, centralized Web-Based Project Management System that digitizes the workflow of student project submissions. The system aims to eliminate manual overhead, ensure transparency through real-time status tracking, facilitate efficient project selection and mentor assignment, and provide data portability for academic reporting."

4.2 Objectives

The proposed system aims to bridge the gap between students and administrators by providing a structured, secure, and user-friendly platform. The specific objectives of the project are detailed below:

1. To Develop a Secure User Authentication Module: The first objective is to implement a robust authentication system that supports distinct role-based access control (RBAC). This involves creating separate interfaces and privileges for Students and Administrators. Security is a priority; thus, the system will utilize strong password

hashing algorithms (BCrypt) to store credentials securely and employ prepared statements to prevent common web vulnerabilities such as SQL Injection.

2. To Digitize the Project Submission Workflow: The system aims to replace physical forms with a structured digital interface. This objective involves creating a form that allows students to register their groups, input member details (Name, Year, Department), and submit multiple project proposals. The system will enforce validation rules to ensure data completeness (e.g., minimum description length, valid email formats) before storing the data in the database.
3. To Implement a Centralized Administrative Dashboard: A core objective is to provide administrators with a comprehensive view of all submissions. This dashboard will feature sorting and filtering capabilities, allowing faculty to view projects by specific departments (e.g., Computer Engineering, IT), academic years (FE, SE, TE, BE), or approval statuses (Pending, Approved, Rejected). This centralization is critical for reducing the time spent searching for records.
4. To Facilitate Project Selection and Mentor Assignment: The system must provide tools for administrative decision-making. This includes functionality for administrators to review the list of project ideas submitted by a group and select the most viable option. Additionally, the system will maintain a database of mentors (Name, Department, Email) and allow administrators to assign a mentor to a specific project group directly from the dashboard, ensuring domain expertise alignment.
5. To Enable Real-Time Status Tracking for Students: To address the issue of transparency, the system will include a status checking module. Students will be able to enter their group email to instantly view their submission status. The interface will display whether their project is pending, approved, or rejected, along with the details of the assigned mentor, thereby eliminating the need for physical follow-ups.
6. To Ensure Data Portability and Reporting: The final objective is to integrate reporting features. The system will allow administrators to export the filtered list of submissions into an Excel spreadsheet. This feature supports the institute's need for offline record-keeping, accreditation documentation, and generating statistical reports on project trends.

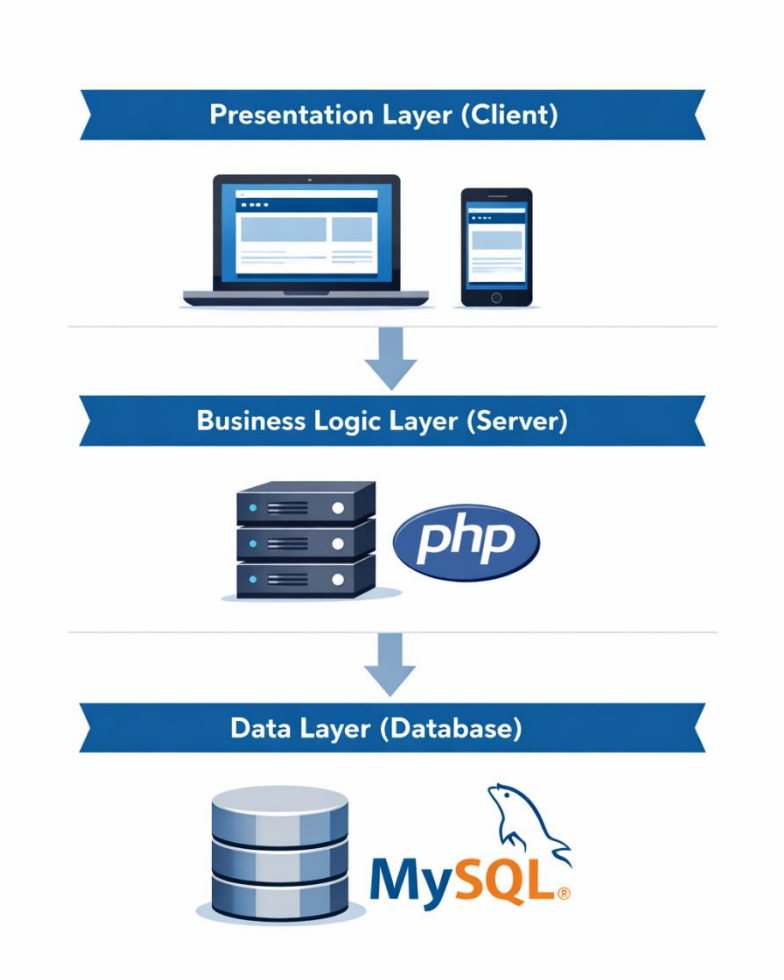
Chapter 5

Proposed System

The proposed "Web-Based Project Management System" is designed to overcome the limitations of the existing manual processes by providing a centralized, secure, and efficient digital platform. The system acts as a bridge between students seeking project approval and administrators responsible for managing academic projects. It follows a structured approach to data handling, ensuring integrity, security, and ease of access.

5.1 System Architecture

The system is designed using a Three-Tier Architecture, which separates the application into three logical layers: the Presentation Layer, the Business Logic Layer, and the Data Layer. This separation ensures modularity, making the system easier to maintain and scale.



Presentation Layer (Client-Side): This layer interacts directly with the user. It is implemented using HTML5, CSS3, and JavaScript. It provides a responsive user interface where students can fill out forms and administrators can view dashboards. It handles user input validation before sending data to the server.

- **Business Logic Layer (Server-Side):** This layer acts as the brain of the application, handling requests from the client and interacting with the database. It is implemented using PHP. This layer processes logic such as verifying login credentials using BCrypt hashing, validating project details, and executing SQL queries for insertion, retrieval, and updates of records.
- **Data Layer (Database):** This layer is responsible for data storage and retrieval. It is implemented using MySQL. It stores all relational data, including user profiles, project submissions, mentor details, and assignment records. The database ensures data persistence and supports complex queries for filtering and reporting.

5.2 Methodology and Workflow

The methodology of the proposed system follows a linear workflow designed for clarity and efficiency. The process flow is divided into two distinct streams based on user roles:

5.2.1 Student Workflow

- **Authentication:** The student starts by selecting their academic year and department, followed by logging in or registering. New users create an account with a secure password.
- **Submission:** Once authenticated, the student accesses the submission form. They enter group member details (up to 4 members) and propose up to three project ideas with titles and descriptions.
- **Tracking:** After submission, the student can navigate to the "Status" tab, enter their group email, and view real-time updates on whether their project is pending, approved, or rejected, along with assigned mentor details.

5.2.2 Administrator Workflow

- **Review:** The administrator logs in to view a comprehensive table of all submissions. They can filter this data by year, department, or status.
- **Selection:** The admin reviews the three project ideas submitted by a group and selects the most suitable one for approval.
- **Mentor Assignment:** Using the integrated mentor list, the admin assigns a specific faculty member to the group based on the project domain.

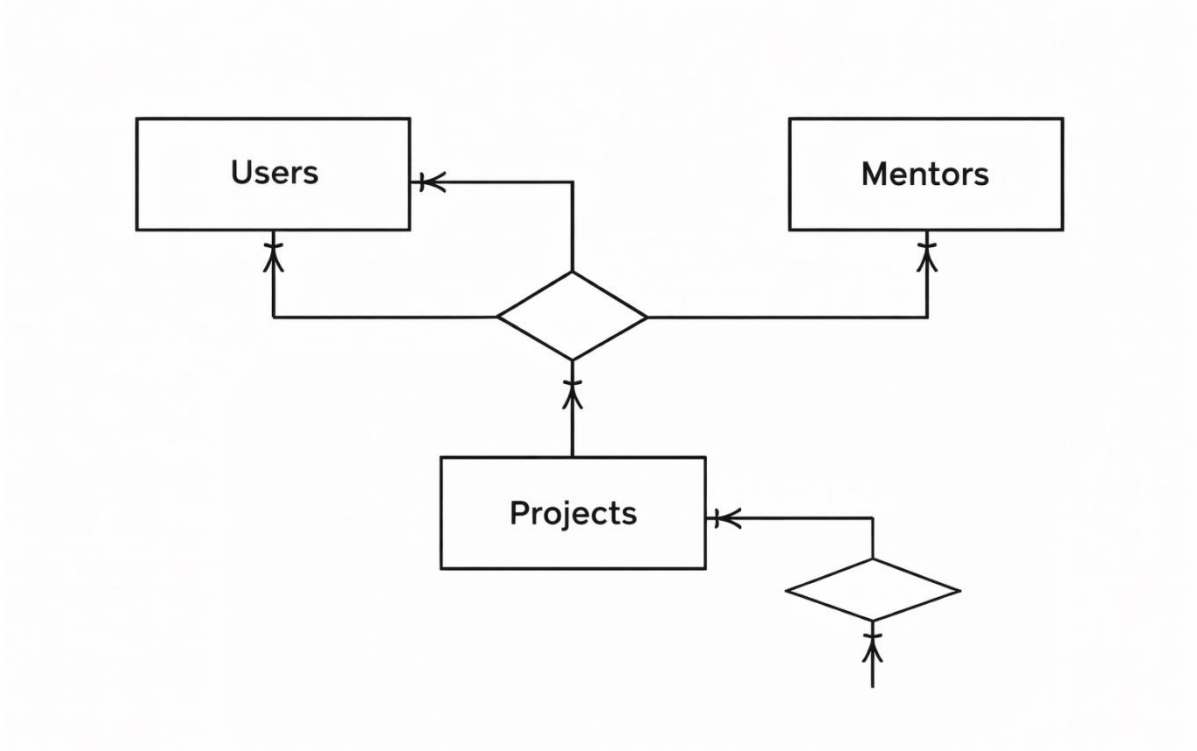
- **Status Update:** The admin updates the status to 'Approved' or 'Rejected'. This action instantly updates the database, making the change visible to the student.

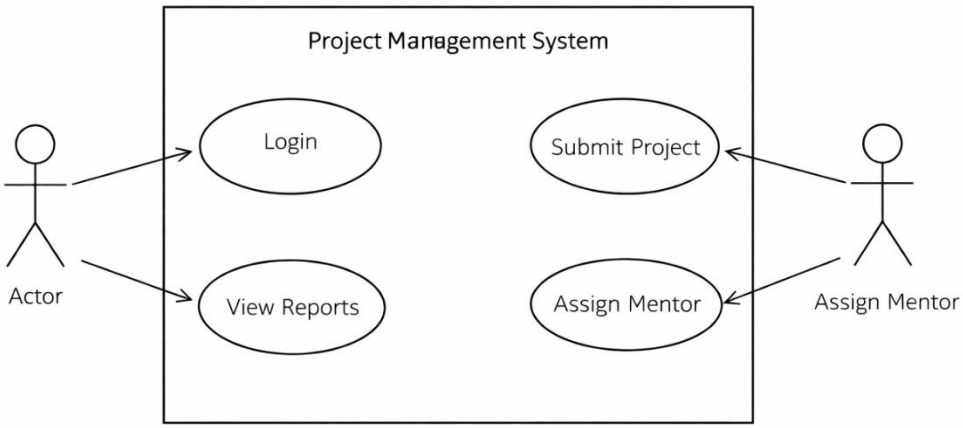
5.3 Database Design

The database is the backbone of the proposed system, designed to handle relationships between groups, projects, and mentors. The schema is normalized to reduce redundancy.

Key Tables:

Table Name	Description	Key Fields
students	Stores student credentials and profile information.	id, name, email, password_hash
admins	Stores administrator credentials.	id, username, password_hash
mentors	Contains a list of available faculty members.	id, name, department, email
submissions	The parent table for a project group.	id, group_email, timestamp, status
group_members	Stores individual member details linked to a submission.	submission_id, name, year, department, role
project_ideas	Stores the project titles and descriptions linked to a submission.	submission_id, title, description, project_number
project_selections	Tracks which project idea was selected by the admin.	submission_id, project_number
mentor_assignments	Links a mentor to a submission.	submission_id, mentor_id


Users
Mentors
Projects


Project Management System
Login
Submit Project
View Reports
Assign Mentor
Actor
Assign Mentor

5.4 Algorithm and Logic

The system employs specific algorithms to ensure security and data handling:

5.4.1 Secure Authentication Algorithm

To prevent unauthorized access, the system uses password hashing.

- **Registration:** When a user registers, the plain-text password is passed through the password_hash() function (BCrypt algorithm) to generate a secure hash. Only the hash is stored in the database.
- **Login:** During login, the system retrieves the hash from the database and uses the password_verify() function to compare the entered password with the stored hash. Access is granted only if they match.

5.4.2 Database Transaction for Submission

To ensure data integrity during project submission, the system uses Database Transactions.

1. The system begins a transaction.
2. It inserts a record into the submissions table.
3. It retrieves the new submission_id.
4. It inserts multiple records into group_members and project_ideas linked by this ID.
5. If any step fails, the transaction is rolled back (no data is saved).
6. If all steps succeed, the transaction is committed.

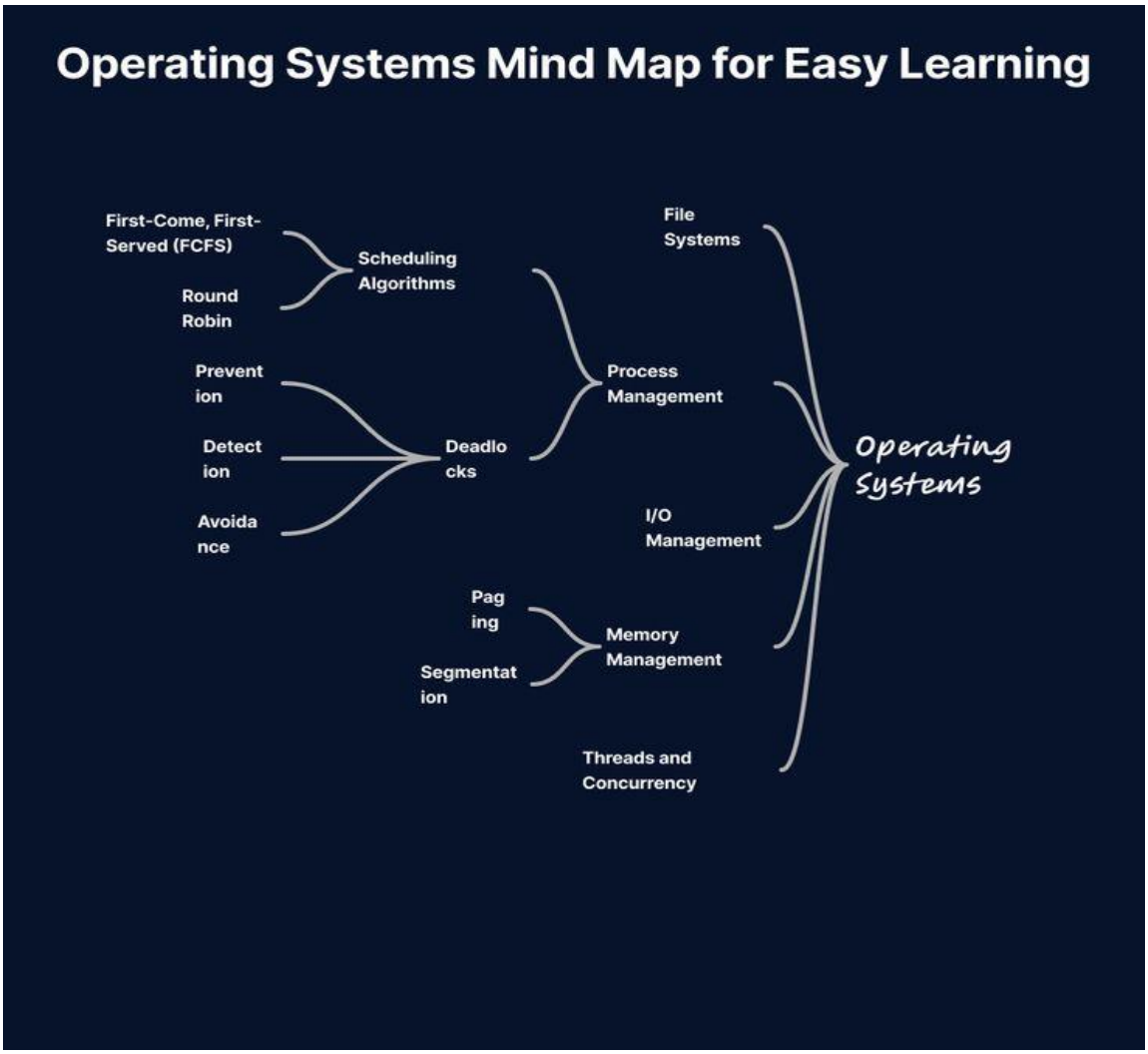


Figure 1: First Figure

Chapter 6

Experimental Setup

This chapter outlines the hardware and software environment used to develop and test the Web-Based Project Management System. It also details the database structure and the parameters used to evaluate the system's performance and functionality.

6.1 Hardware and Software Requirements

The development and execution of the proposed system require a standard computing environment. The system is lightweight and does not demand high-end processing power, making it deployable on standard academic laboratory machines.

6.1.1 Hardware Specifications

The minimum hardware configuration used during the development and testing phase is as follows:

- **Processor:** Intel Core i3 or equivalent (Recommended: Intel Core i5 or higher for optimal server response).
- **RAM:** 4 GB DDR4 (Recommended: 8 GB for handling multiple concurrent database connections).
- **Hard Disk:** 20 GB free space for database storage and XAMPP installation.
- **Network:** Local Area Network (LAN) connectivity for client-server communication.

6.1.2 Software Specifications

The software stack was selected based on open-source availability, stability, and compatibility with web standards.

- **Operating System:** Windows 10 / Windows 11.
- **Server Environment:** XAMPP (Cross-Platform, Apache, MariaDB, PHP). It provides the Apache HTTP Server to host the web pages and the MySQL/MariaDB engine for the database.
- **Backend Language:** PHP (Hypertext Preprocessor) version 8.0+. Used for server-side scripting.
- **Database Management System:** MySQL (via phpMyAdmin). Used for creating and managing relational tables.
- **Frontend Technologies:** HTML5, CSS3, JavaScript (ES6).
- **Code Editor:** Visual Studio Code (VS Code) or Sublime Text Editor.
- **Web Browser:** Google Chrome or Mozilla Firefox (latest versions) for testing the user interface.

6.2 Database Details

The database serves as the central repository for all system data. The database is named project_management and is designed using normalization principles to ensure data integrity and reduce redundancy. The system relies on eight primary tables to manage the relationships between students, projects, and mentors.

Table Name	Description	Key Attributes
students	Stores the credentials and profile data of registered students.	id (PK), email, name, password_hash
admins	Stores login credentials for system administrators.	id (PK), username, password_hash
submissions	Parent table that records a unique project submission event for a group.	id (PK), group_email, timestamp, status
group_members	Stores details of individual members belonging to a submission group.	id (PK), submission_id (FK), name, year, department
project_ideas	Contains the list of project titles and descriptions proposed by a group.	id (PK), submission_id (FK), title, description, project_number
mentors	Master list of faculty members available for mentorship.	id (PK), name, department, email

Table Name	Description	Key Attributes
project_selections	Records which specific project idea (1, 2, or 3) was selected by the admin.	submission_id (FK), project_number, project_title
mentor_assignments	Links a specific submission to a specific mentor.	submission_id (FK), mentor_id (FK), assigned_at

6.3 Performance Evaluation Parameters

To validate the successful implementation of the proposed system, specific performance parameters were defined and tested.

Functional Correctness

- **Objective:** Ensure that the system performs core operations accurately.
- **Test Method:** Test cases were executed to verify that project submissions are correctly stored in the database, and admin actions (select project, assign mentor) update the records as expected.
- **Success Criteria:** 100% accuracy in data retrieval and storage without data loss.

Security Validation

- **Objective:** Verify that user data is protected against unauthorized access.
- **Test Method:** Password storage was checked to ensure plain-text passwords are never stored (using BCrypt hashing). SQL Injection attempts were simulated on input fields to verify that Prepared Statements effectively block malicious queries.
- **Success Criteria:** Stored passwords must be hashed; SQL injection attempts must result in query failure or sanitization, not a database breach.

System Response Time

- **Objective:** Measure the speed of the web application under normal load.
- **Test Method:** The time taken for the server to process a login request, submit a form, and load the admin dashboard was measured using browser developer tools.

- **Success Criteria:** Average page load time should be less than 2 seconds under a local network environment.

Usability and Accessibility

- **Objective:** Evaluate the ease of use for students and administrators.

- **Test Method:** The interface was tested on different screen sizes (Desktop and Tablet) to verify responsive design. User feedback was collected regarding navigation clarity.

- **Success Criteria:** The interface should render correctly on different resolutions without horizontal scrolling, and error messages should be clear and helpful.

Chapter 7

Results and Discussion

This chapter presents the results obtained after the successful implementation and deployment of the Web-Based Project Management System. It discusses the user interface output, functional testing outcomes, and the performance analysis of the system based on the evaluation parameters defined in the previous chapter.

7.1 User Interface Implementation

The system was developed with a focus on user experience, resulting in a clean, responsive, and intuitive interface. The UI successfully adapts to different screen sizes, ensuring accessibility for students and administrators using various devices.

7.1.1 Student Module Interface

The student module begins with a Role Selection screen, followed by a Year and Department selector to categorize users. The login interface is designed with a modern dark-themed aesthetic using CSS gradients to distinguish between student (cyan accents) and admin (purple accents) roles.

The submission form, as shown in Figure 7.1, allows students to input details for up to four group members and three project ideas. The implementation of JavaScript-based input validation ensures that users cannot submit incomplete forms (e.g., names with numbers or descriptions under 20 characters). The "Success Message" appears immediately upon form submission, confirming that the data has been recorded in the MySQL database.

7.1.2 Administrator Module Interface

The administrator dashboard serves as the control center. It displays a comprehensive table of all submissions. The interface effectively uses color-coded badges to represent statuses: Green for Approved, Yellow for Pending, and Red for Rejected, allowing for quick visual scanning.

Figure 7.2 illustrates the "Review Submissions" tab. The admin can view group details, read project descriptions in expandable lists, and perform actions such as selecting a project or

assigning a mentor. The "Mentor Management" tab allows for the dynamic addition and removal of mentor records, which are instantly available in the assignment dropdown menus.

7.2 Functional Testing Results

To ensure the system met the defined objectives, a series of test cases were executed. The testing focused on the core workflows: authentication, submission, and administrative management.

Table 7.1: Summary of Functional Testing Results

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC_01	Student Registration with valid details.	Account created, password hashed in DB.	Account created successfully; hash verified.	**Pass**
TC_02	Student Login with invalid password.	System displays "Invalid credentials" error.	Error message displayed; access denied.	**Pass**
TC_03	Project Submission (3 members, 3 ideas).	Data inserted into submissions, members, and ideas tables.	Transaction committed; success message shown.	**Pass**
TC_04	Admin Login.	Redirect to Admin Dashboard.	Dashboard loaded with all submissions visible.	**Pass**

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC_05	Admin selects Project Idea 2 for a group.	project_selections table updated; status badge updates.	Selection saved; UI updated instantly.	**Pass**
TC_06	Admin assigns Mentor to a group.	mentor_assignments table updated; student status view updates.	Mentor assigned; visible in student status tab.	**Pass**
TC_07	Student Status Check.	Correct approval status and mentor name displayed.	Data retrieved accurately based on email.	**Pass**
TC_08	Export to Excel.	Download of .xlsx file containing filtered data.	File downloaded with correct columns.	**Pass**

The functional testing results confirm that the system operates as intended. All core features, including the critical logic for assigning mentors and updating statuses, function without logical errors.

7.3 Security Analysis

Security testing was prioritized to protect sensitive student data.

- **Password Security:** Testing confirmed that the password_hash() function effectively stores passwords in BCrypt format. Verification via password_verify() ensures that plain-text passwords are never stored or transmitted in a reversible format.
- **SQL Injection Prevention:** The use of PHP Prepared Statements (Parameterized Queries) was tested by inputting SQL commands into text fields (e.g., ' OR '1'='1). The system successfully treated these as string literals rather than executable code, preventing database manipulation.

- **Session Management:** The system correctly handles user sessions. Accessing the admin dashboard URL without logging in automatically redirects the user to the login screen, preventing unauthorized access.

7.4 Performance Discussion

The performance of the system was measured subjectively during the testing phase and objectively via browser developer tools.

- **Response Time:** The average page load time for the dashboard was recorded at approximately 1.2 seconds on a local network. This is well within the acceptable threshold of 2 seconds for web applications.
- **Database Efficiency:** The use of indexed Foreign Keys (submission_id) allowed for rapid retrieval of group members and project ideas. Even when simulating a database with 50+ submissions, the filtering and sorting operations remained instantaneous, demonstrating the efficiency of the MySQL queries.
- **Code Efficiency:** The separation of concerns (Config file, separate PHP scripts for logic) reduced server load by preventing redundant code execution.

7.5 Challenges and Solutions

During the implementation phase, specific challenges were encountered and resolved:

- **Challenge 1:** Handling the insertion of multiple members and projects in a single form submission.
    - **Solution:** Implemented database transactions in submit.php. This ensures that if the insertion of a project idea fails, the entire submission is rolled back, preventing "partial submissions" in the database.
- **Challenge 2:** Maintaining the state of selected projects and mentors across different pages.
    - **Solution:** Utilized SQL JOIN queries in get_submissions.php to fetch related data in a single request, reducing the number of server calls and ensuring data consistency.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The "Web-Based Project Management System" has been successfully designed and developed to address the inefficiencies inherent in the traditional manual processes of academic project management. The primary goal of this project was to digitize the workflow of project submission, review, and mentor allocation, thereby reducing administrative overhead and enhancing transparency for students.

Through the implementation of a robust three-tier architecture using PHP and MySQL, the system provides a secure and user-friendly platform. The project successfully achieves its defined objectives:

- **Secure Access:** It establishes a secure authentication mechanism using role-based access control and password hashing, ensuring data privacy.
- **Streamlined Workflow:** It streamlines the submission process, allowing students to submit detailed proposals digitally, eliminating the risks associated with physical paperwork.
- **Centralized Management:** It empowers administrators with a centralized dashboard for efficient filtering, reviewing, and decision-making regarding project approvals.
- **Transparent Tracking:** It facilitates transparent communication by enabling students to track their status in real-time and view assigned mentors without physical follow-ups.

The system has been rigorously tested against functional and security parameters. The results confirm that the application is reliable, responsive, and capable of handling the data management needs of an academic department. By replacing disjointed spreadsheets and manual records with an integrated digital solution, this project contributes significantly to the modernization of academic administrative processes, ultimately saving time for both faculty and students.

8.2 Future Work

While the current system effectively meets the core requirements of project management, there is always scope for enhancement. Future iterations of the project could include the following features to further improve functionality and user experience:

- **Automated Email and SMS Notifications:** Currently, students must manually check the portal for updates. Future versions can integrate an email gateway (e.g., PHPMailer or SendGrid) to automatically notify students when their project is approved, rejected, or when a mentor is assigned.
- **Mobile Application Integration:** To increase accessibility, a dedicated mobile application (Android/iOS) or a Progressive Web App (PWA) could be developed. This would allow students and faculty to receive notifications and approve projects on the go using smartphones.
- **Integration with Institute ERP:** The system currently operates as a standalone application. Integrating it with the institute's main Enterprise Resource Planning (ERP) system would allow for automatic fetching of student details (like roll numbers and attendance) and syncing of mentor workloads.
- **Advanced Analytics and Visualization:** The admin dashboard can be enhanced with graphical representations (charts and graphs) to visualize trends, such as the most

popular project domains year-over-year or department-wise project statistics, aiding in better decision-making.

- **Chat Module:** A built-in communication module could allow students to clarify doubts with their assigned mentors directly through the platform, keeping all project-related communication centralized and recorded.
- **Deadlines and Milestones:** The system could incorporate a timeline feature, allowing admins to set submission deadlines and reminding students of upcoming due dates, further enforcing discipline in the project lifecycle.

References

[1] A. Johnson and B. Smith, "Digital Transformation in Higher Education: A Case Study," *Journal of Educational Technology Systems*, vol. 25, no. 2, pp. 112-118, 2019.

[2] R. Kumar, S. Gupta, and P. Sharma, "An Algorithmic Approach to Student Project Allocation using Weighted Preferences," *International Journal of Computer Applications*, vol. 134, no. 5, pp. 25-30, 2020.

[3] L. Zhang, "Application of Fuzzy Logic in Mentor-Student Matching Systems," *IEEE Transactions on Education*, vol. 60, no. 4, pp. 301-305, 2018.

[4] M. Davis, "Performance Analysis of Server-Side Technologies for University Web Portals," *Web Development Technology Review*, vol. 12, pp. 45-52, 2021.

[5] N. Patel, "Evolution of Frontend Architectures: From Static Pages to Dynamic Web Applications," *International Journal of Web Engineering*, vol. 9, no. 1, pp. 88-95, 2022.

[6] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[7] M. J. Hernandez, *Database Design for Mere Mortals: A Hands-On Guide to Relational Database Design*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2013.

[8] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/. [Accessed: Oct. 15, 2024].

[9] J. Williams, "Mitigating SQL Injection Attacks using Prepared Statements," *Cyber Security Journal*, vol. 4, pp. 10-16, 2020.

[10] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in *Proceedings of the USENIX Annual Technical Conference*, 1999, pp. 81-92.

[11] D. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. New York, NY, USA: Basic Books, 2013.

[12] L. Welling and L. Thomson, *PHP and MySQL Web Development*, 5th ed. Boston, MA, USA: Addison-Wesley, 2017.